HACKTOBER FEST
Four Final Problem Statements

Open Source AI Hackathon | Organized by Elevate

This document gives a simple, self-contained explanation of the four final challenge tracks. Each team must select exactly one challenge and build its final project around that challenge.

1. Best Open-Source AI Project

Build an original and useful project in which open-source or open-weight AI is a central part of the system. The project should demonstrate meaningful engineering rather than simply wrapping an existing AI API.

What the team needs to build

- Create a real-world or meaningful application with a clearly defined problem and target users.
- Use an open-source or open-weight AI component as an important part of the solution.
- Possible approaches include open-weight LLMs or SLMs, AI agents, agent skills, model harnesses, RAG systems, vision-language models, or combinations of these.
- Show how the selected AI component interacts with the rest of the application and why it is necessary.
- Provide a functional final implementation and publish the project in a public GitHub repository under an appropriate open-source license.

Important expectations

- The AI component should contribute substantially to the core functionality.
- A basic chatbot, superficial API wrapper, or project where AI is only an optional add-on is not the intended solution.
- Teams should demonstrate architecture, data flow, implementation choices, and meaningful technical work.
- If submitting an agent skill, it should follow the applicable Agent Skill Open Standard.
- If submitting a model-harness project, the team should provide an original implementation or meaningful changes to an existing open-source harness.

Expected final outcome

A complete working project that solves a defined problem and clearly demonstrates how open-source AI enables the solution.

2. Best Use of Gemma 4 / Gemma 4 Open-Source

Build a functional project that makes meaningful use of Google's lightweight Gemma 4 open-weight model. The project should demonstrate a practical use of Gemma 4 rather than functioning as a simple API demonstration.

What the team needs to build

- Choose a clear problem that can benefit from Gemma 4.
- Use Gemma 4 meaningfully in the core workflow of the application.
- The project may work with text, images, or multimodal inputs where appropriate.
- Build a functional prototype or system that demonstrates the model's contribution to the solution.
- Clearly explain the input, processing flow, model interaction, and final output.

Important expectations

- Gemma 4 should have a meaningful role in the system rather than being included only to satisfy the challenge.
- The project should go beyond a superficial wrapper around the model.
- Teams should explain why Gemma 4 was selected and what part of the problem it solves.
- The final system should demonstrate actual engineering, integration, and a clear user-facing or technical outcome.

Possible directions

Teams may explore multimodal assistants, productivity tools, learning applications, creative tools, document understanding, community-focused applications, or other focused AI systems where Gemma 4 provides a meaningful capability.

Expected final outcome

A working project that clearly demonstrates a useful application of Gemma 4 and explains how the model contributes to the system.

3. VYOM+ — End-to-End AI-Powered GST Invoice Intelligence System

Build an end-to-end invoice intelligence system for VYOM+ that can accept different invoice and transaction document formats, process them using an appropriate AI pipeline, and convert them into accurate, validated, machine-readable financial records.

Input formats

- Excel (.xlsx)
- CSV
- PDF
- JPEG / JPG
- PNG

What the system should do

- Identify the type of input and route it through an appropriate processing pipeline.
- For Excel/CSV, organize the available information into a clean and structured tabular format.
- For PDFs and images, extract the relevant invoice and GST information and convert it into structured records.
- Extract important invoice, GST, tax, financial, and line-item information.
- Validate extracted information and identify or handle inconsistent or uncertain data where possible.
- Return standardized machine-readable output such as JSON and/or structured tabular data.
- Provide a simple interface where an evaluator can upload documents and inspect the resulting structured information.

Main technical focus

The challenge should primarily address reliable document intelligence, with particular attention to improving handwritten GST invoice processing while maintaining strong performance on printed and digitally generated invoices.

AI and technology

- Teams may use open-source or appropriately licensed OCR systems, vision-language models, document-understanding models, LLMs, and related AI components.

- The solution should be designed as a complete processing pipeline rather than only a text-recognition demo.
- The system should be capable of turning real-world invoice documents into accurate and usable financial records.

Expected final outcome

A working document-intelligence system that accepts supported invoice formats and produces validated, structured financial data suitable for downstream accounting workflows.

4. VYOM+ — Intelligent Voucher Classification Using Open-Source LLMs

Build an AI system that determines the appropriate accounting voucher category from already-structured transaction information. The key challenge is reasoning over multiple fields and understanding the accounting meaning of a transaction.

Important constraint

This is NOT an OCR or invoice-extraction challenge. Participants will receive an Excel dataset containing the relevant information for each transaction. The voucher type will intentionally be missing.

Input dataset

- Participants will receive an Excel (.xlsx) file containing multiple invoices or financial transactions.
- Each row represents a transaction/document and contains available structured fields.
- Possible fields include seller/supplier, buyer/customer, invoice number and date, item descriptions, quantities, taxable value, GST, discounts, freight, payment information, currency, import/export details, payroll information, debit/credit information, return information, order references, delivery information, and other metadata.
- The voucher-type column will not be provided.

Target voucher categories

- Purchase
- Sales
- Purchase Return / Debit Note
- Sales Return / Credit Note
- Payment
- Receipt
- Contra
- Journal
- Salary / Payroll
- Attendance
- Purchase Order
- Sales Order
- Receipt Note
- Delivery Note
- Rejection In
- Rejection Out
- Stock Journal
- Physical Stock
- Material In

- Material Out
- Job Work In Order
- Job Work Out Order
- Import
- Export
- Expense
- Advance / Prepayment
- Other / Miscellaneous

What the system should do

- Read and process the provided Excel dataset.
- Understand the relationship between multiple transaction fields instead of relying on a single keyword.
- Use an open-source/openly available LLM or SLM, or an appropriate AI architecture, as the primary intelligence layer.
- Predict one appropriate voucher category for each transaction.
- Produce structured output that can be evaluated programmatically.
- Handle missing, incomplete, or ambiguous transaction information as robustly as possible.
- Test the system on previously unseen records and provide a reproducible evaluation method.

Example output

Minimum output can follow a structure such as: { "invoice_number": "INV-2026-1042", "voucher_type": "Purchase" }. Teams may additionally provide a confidence score and a short explanation for the prediction.

Core technical challenge

The system must distinguish semantically similar transaction types by reasoning over the complete transaction context. Examples include Purchase vs Sales, Purchase Return vs Sales Return, Salary/Payroll vs other transactions, Contra vs ordinary Payment/Receipt, Journal vs conventional Purchase/Sales events, inventory movement vs Purchase/Sales, and Import/Export vs other transaction types.

Open-source AI requirement

- The primary classification intelligence should use an open-source or openly available LLM/SLM or suitable AI architecture.
- Possible approaches include local inference, quantized models, LoRA/QLoRA fine-tuning, embeddings, structured-output generation, RAG, agentic workflows, or hybrid ML + LLM systems.
- Example model families include Gemma, Llama, Qwen, Mistral, Phi, and other appropriately licensed open models.
- A proprietary API should not be the primary classification engine.

Evaluation

- The solution may be evaluated against a hidden dataset with known ground-truth voucher types.
- Evaluation may consider accuracy, precision, recall, F1-score, per-category performance, handling of ambiguous or incomplete records, output consistency, inference speed, and computational efficiency.
- Special attention may be given to distinguishing semantically similar transaction categories.

Expected final outcome

A working AI classifier that takes structured transaction data as input and produces the appropriate voucher category for each transaction, forming a potential bridge between invoice extraction and automated voucher creation.

General Rules for All Four Challenges

- Each team must select exactly one of the four problem statements.
- A team working on one challenge cannot submit or compete in the other three challenge tracks.
- The final project must contain meaningful technical implementation and should not be only a conceptual proposal.
- Open-source AI should have a meaningful role in the project wherever required by the selected challenge.
- For the qualifier round, teams submit a README-only GitHub repository; implementation happens during the final hackathon.
- For the final round, teams are expected to demonstrate a working implementation of the proposed system.

Qualifier Reminder

The qualifier round runs from 3–8 October. The GitHub repository must contain only README.md. The README should explain the problem, proposed solution, target users, selected open-source AI technology, AI's role, architecture, data flow, technology stack, implementation plan, expected output, scalability, dependencies, and expected challenges. No implementation code, datasets, notebooks, binaries, or generated files should be included in the qualifier repository.